Algorithm for file updates in Python

Project description

In this scenario, I was tasked to write a Python script to automate the process of updating access permissions for certain employees based on a remove list of IP addresses. I developed a simple algorithm to remove IP addresses that were no longer allowed to access personal patient records from the allow list.

Open the file that contains the allow list

First, I used a `with` statement to open the file that contains the allow list, opening the file with the `"r"` argument for reading:

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First line of `with` statement
with open(import_file, "r") as file:
```

Read the file contents

Inside the with statement, I read the contents of the file into the string `ip_addresses`, which I then printed out:

```
# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
  ip_addresses = file.read()

# Display `ip_addresses`
print(ip_addresses)
```

Convert the string into a list

In order to manipulate the individual strings from the file, I converted the string `ip_addresses` into a list by using the `split()` method:

```
# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Display `ip_addresses`
print(ip_addresses)
```

Iterate through the allow list

Next, I iterated through the allow list using a `for` loop, displaying each IP address:

```
# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

  # Display `element` in every iteration
  print(element)
```

Remove IP addresses that are on the remove list

Inside my `for` loop, I then added a conditional to check each IP address in the allow list against the IP addresses in the remove list and remove any that were on the remove list:

```
# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,
  if element in remove_list:

    # then current element should be removed from `ip_addresses`
    ip_addresses.remove(element)

# Display `ip_addresses`
print(ip_addresses)
```

Update the file with the revised list of IP addresses

Finally, I converted the list `ip_addresses` back into a string using the `join()` method and overwrote the contents of the allow-list file with another `with` statement, opening the file with the `"w"` argument for writing:

```
# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = "\n".join(ip_addresses)

# Build `with` statement to rewrite the original file
with open(import_file, "w") as file:

  # Rewrite the file, replacing its contents with `ip_addresses`
  file.write(ip_addresses)
```

Summary

In conclusion, using foundational Python concepts, I was able to develop and implement an algorithm for checking an allow list of IP addresses against a remove list and updating the allow-list file accordingly. I used file I/O, iterative and conditional statements, and special string and list methods.